

User Manual

Log Analyzer & Report Generator (LARG)

Prepared by Group 6

Version 1.0

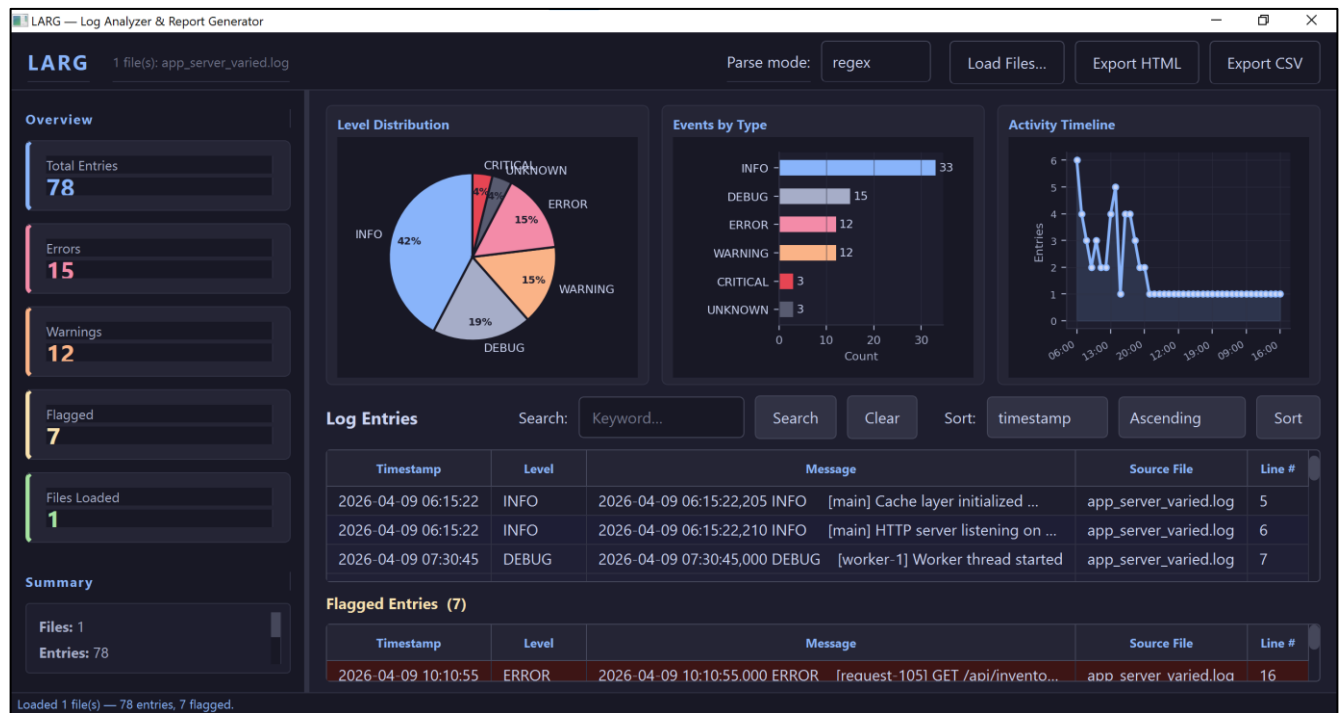
Team Members – Group 6: Amanda, Mark, Lian and Lian

1. Purpose

The Log Analyzer & Report Generator (LARG) is a standalone desktop application built with Python and PyQt6 for analyzing .log and .txt files. The application allows users to load one or more files, parse log entries, filter and sort results, view summary analytics, and export reports in HTML or CSV format. [Github repo](#).

2. Main Features

- Import one or more .log or .txt files.
- Use Template, Generic, or Regex parsing modes.
- Search by keyword and exclude keyword.
- Filter by log level and date range.
- Sort results by timestamp, level, message, or source file.
- View analytics such as total entries, counts by level, top recurring messages, time buckets, and flagged entries.
- Export results to HTML and CSV.



3. System Requirements

- Operating system: Windows 10 or later recommended.
- Python 3.10 or later.
- Required Python packages: PyQt6 and matplotlib.
- Optional tool for executable build: PyInstaller.

4. Project Files

File or Folder	Purpose
main.py	Application entry point.
ui_workflow.py	PyQt frontend and user workflow.
ui_constants.py / ui_helpers.py / ui_widgets.py	UI resources and helper modules.
parsing_searching.py	Parsing, searching, and sorting logic.
analytics_reporting.py	Analytics and export/report generation.
sample_logs/	Sample files for testing.
test_analytics_reporting.py / test_exporters.py / test_parsing_searching.py	Project test files.
LogAnalyzer.spec	PyInstaller build specification file.

5. Installation

Install Python packages in a terminal or command prompt:

```
pip install pyinstaller
pip install -r requirements.txt
```

6. Running the Application

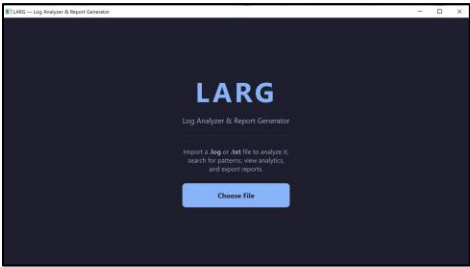
To start the application, open a terminal in the project folder and run:

```
python main.py
```

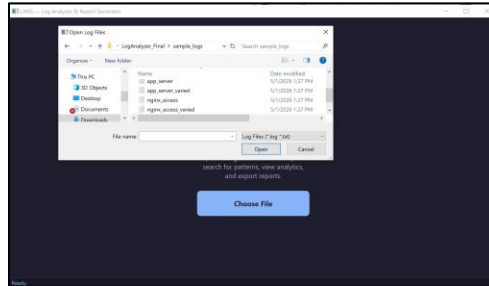
After startup, the main dashboard opens. From there, users can select files, choose a parsing mode, apply filters, view charts and log entries, and export results.

7. User Workflow

- 7.1 Open the application.



- 7.2 Select one or more log files, or select a folder containing log files.



- 7.3 Choose a parsing mode. – not required for the first time – *default template*
- 7.4 Review imported entries in the Log Entries tab.
- 7.5 Apply filters such as keyword, exclude keyword, level, or date range. Also, you can sort the results as needed.
- 7.6 Review dashboard metrics, charts, and summary text.
- 7.7 Export the results to HTML or CSV.

8. Parsing Modes

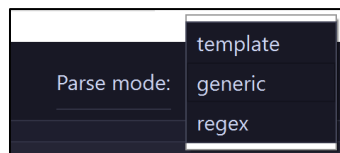
- **Template Mode:** Use this mode when log lines follow the default format: YYYY-MM-DD HH:MM:SS LEVEL message.
- **Generic Mode:** Use this mode for unstructured files. Each non-empty line is stored as a plain text message.
- **Regex Mode:** Use this mode when logs follow a custom structure. Supply a regular expression with named groups such as timestamp, level, and message.

Example Template line:

```
2026-04-01 10:01:22 ERROR Failed to connect to database
```

Example Regex pattern:

```
^(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\s+(?P<level>\w+)\s+(?P<message>.+)$
```



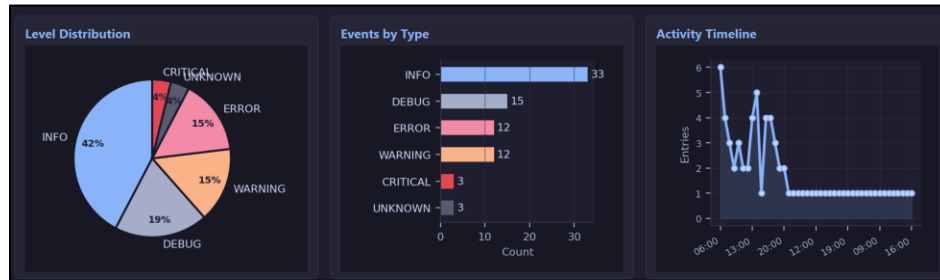
9. Search, Filter, and Sort

- **Keyword:** returns only entries whose message contains the specified text.
- **Exclude keyword:** removes entries whose message contains the specified text.
- **Log level:** limits results to one severity, such as INFO, WARN, ERROR, DEBUG, or CRITICAL.
- **Date range:** accepts start and end dates in YYYY-MM-DD format.
- **Sorting:** available by timestamp, level, message, and source file.

Log Entries		Search: Stack	Search	Clear	Sort: timestamp	Ascending	Sort
Timestamp	Level	Message	Source File	Line #			
2026-04-09 10:10:55	ERROR	2026-04-09 10:10:55,010 ERROR [request-105] Stack trace:	app_server_varied.log	17			

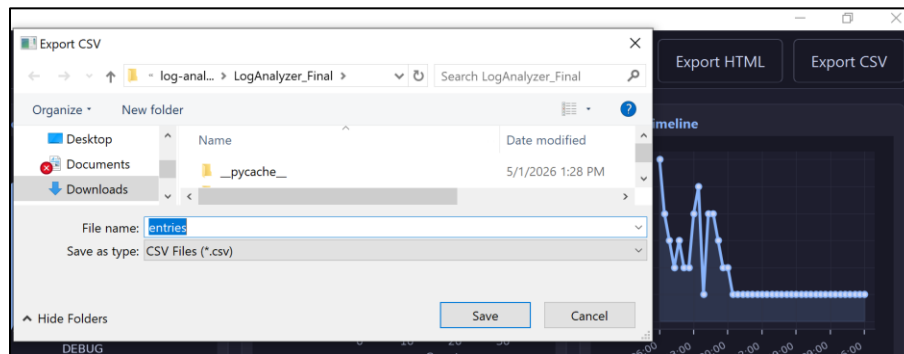
10. Analytics Dashboard

The dashboard displays high-level metrics and charts to help users understand the current result set. Typical analytics include total entries, counts by log level, top recurring messages, time buckets, and flagged entries.



11. Exporting Results

- **HTML Export:** creates a formatted report suitable for viewing in a web browser or attaching to documentation.
- **CSV Export:** creates a structured file that can be opened in spreadsheet software for further analysis.



12. Building the Executable

To rebuild the Windows executable after making code changes:

```
cd path/to/log-analyzer-project/LogAnalyzer_Final
```

```
python -m PyInstaller LogAnalyzer.spec --clean
```

When the build is complete, the executable is created in the dist folder. The build folder contains intermediate files and can usually be ignored.

13. Troubleshooting

- If the application does not start, verify that all required packages are installed.

- If no data appears after loading a file, confirm the file extension is .log or .txt and that the selected parsing mode is appropriate.
- If date filtering fails, verify the format is YYYY-MM-DD.
- If Regex mode returns poor results, review the named groups in the pattern.
- If exports do not appear, confirm you selected a valid save location.

14. Testing

The repository includes test files for parsing/searching, analytics/reporting, and exporters. These tests should be run after major changes to help confirm that the application still behaves as expected.

15. Support and Maintenance

For team development, contributors should pull the latest code, work in a feature branch, and submit changes through GitHub for review. This helps maintain stability in the main branch and reduces integration problems.